

Semana 3 - Endpoints para recebimento e consulta de logs.

Documentação do Backend — Semana 3

O que foi feito

Nesta semana foram criados os endpoints para recebimento e consulta de logs do sistema operacional. O modelo LogEntry foi implementado mapeando a tabela logs já existente no banco. As rotas foram extraídas do app.py para um Blueprint dedicado (app/routes/logs.py), seguindo o mesmo padrão arquitetural adotado no routes/discovery.py. O endpoint POST /api/metrics recebeu um aviso de log sinalizando que o campo data.logs ainda não é processado separadamente, pendente de alinhamento com a equipe do agente.

Arquivos criados ou alterados

app/routes/logs.py · (novo)

Blueprint dedicado às rotas de logs. Criado para seguir o mesmo padrão arquitetural do routes/discovery.py, mantendo o app.py limpo e focado na inicialização.

Melhorias em relação à versão anterior:

- host_id validado como numérico — retorna 400 se vier como string inválida
- log_type usa "system" como valor padrão se não informado
- Rotas movidas para Blueprint separado, fora do app.py

app/routes/_init_.py · (alterado)

Atualizado para exportar a nova função register_log_routes:

```
1 from .discovery import register_discovery_routes from .logs import register_log_routes
2
3 all = ['register_discovery_routes', 'register_log_routes']
```

app/app.py · (alterado)

Três mudanças aplicadas:

1. Endpoints de logs removidos do app.py — movidos para routes/logs.py
2. register_log_routes registrado junto com register_discovery_routes
3. app.logger.warning() adicionado no POST /api/metrics para sinalizar que data.logs

não é processado

Trecho do warning adicionado:

```

1 app.logger.warning(
2
3 ' [/api/metrics] Campo "data.logs" recebido mas nao processado. '
4
5 'Contrato de separacao entre metricas e logs pendente com equipe do agen
6
7 )

```

app/models/log_entry.py · (novo)

Modelo SQLAlchemy para a tabela logs — já existente no banco (criada pela Beatriz). Nenhuma tabela nova foi criada.

Colunas:

- id — chave primária (BIGSERIAL)
- host_id — FK para host (obrigatório)
- timestamp — momento do log (TIMESTAMPTZ, obrigatório)
- log_type — tipo do log, ex: "auth", "system" (VARCHAR 50, padrão: "system")
- raw_line — linha bruta do log (TEXT, obrigatório)
- parsed_data — campos extraídos pelo parsing (JSONB, nullable — preenchido na Semana 4 pelo responsável pelo BD)

app/models/_init_.py · (alterado)

Atualizado para registrar o LogEntry. A função registrar_modelos(db) agora retorna cinco modelos:

```

1 return HostModel, AgentModel, DiscoveryModel, MetricModel, LogEntryModel

```

app/models/host.py · (alterado)

Adicionado o relacionamento 1:N com LogEntryModel:

```

1 logs = db.relationship('LogEntryModel', back_populates='host', lazy='dyn

```

Endpoints implementados

POST /api/logs

Recebe logs do sistema operacional enviados pelo agente e persiste na tabela logs.

Payload esperado (contrato definido com equipe do agente — tarefa 3.1):

```

1 {
2
3 "host_id": 1,
4
5 "timestamp": "2026-04-21T14:35:00Z",

```

```
6
7  "log_type":  "auth",
8
9  "raw_line":  "Apr 21 14:35:00 server sshd[123]: Failed password for root from
10
11 }
```

Campos obrigatórios: host_id, timestamp, raw_line

Campos opcionais: log_type (padrão: "system"), parsed_data

Validações aplicadas:

- Retorna 400 se host_id, timestamp ou raw_line estiverem ausentes
- Retorna 400 se host_id não for numérico
- Retorna 404 se o host não existir no banco
- Aceita timestamp ISO 8601 ou Unix float

Resposta:

```
1 {
2
3  "message":  "Log salvo com sucesso", "log_id":  1,
4
5  "host_id":           1, "log_type": "auth"
6
7 }
```

Código HTTP: 201 Created

GET /api/logs

Consulta logs de um host com paginação e filtro opcional por tipo.

Parâmetros (query string):

- host_id — obrigatório
- limit — padrão 20, máximo 100
- offset — padrão 0
- log_type — opcional (ex: "auth")

Exemplo:

```
1 GET /api/logs?host_id=1&log_type=auth&limit=10
```

Resposta:

```
1 {
2
3  "logs": [...],
```

```
4  "total":      87,  
5  "limit":     10,  
6  "offset":    0,  
7  "host_id":   1,  
8  "hostname":  "teste-ubuntu",  
9  "log_type":  "auth"  
10 }  
11
```

Código HTTP: 200 OK

Observações técnicas

- A tabela logs já existia no banco — o backend não criou nenhuma tabela nova nesta semana.
- O campo `parsed_data` é salvo como NULL nesta semana para todos os logs recebidos. O parsing de logs SSH será implementado na Semana 4 com a função `parse_ssh_log()`.
- Os logs chegam embutidos no payload de métricas (`data.logs[]`). Um `app.logger.warning()` foi adicionado no POST `/api/metrics` para registrar essa pendência até o alinhamento com a equipe do agente.
- A tarefa 3.5 (testar com logs reais do agente) será executada assim que o agente estiver disponível no ambiente de testes compartilhado.

Também concluído nesta semana

Discovery — suporte a dois ambientes (físico e VM)

Foram analisados e documentados os contratos JSON reais enviados pelo agente, identificando dois formatos distintos: um para máquina física (Linux Mint) e outro para ambiente virtualizado (Ubuntu 24.04 em VMware). O endpoint POST `/api/discovery` foi atualizado para suportar os dois formatos de forma tolerante, extraindo corretamente campos como `cpu_vcpus`, `memory_total_gb` e frequência de CPU independente da estrutura do payload. O modelo `HostDiscovery` foi expandido com quatro novas colunas (`os_name`, `os_version`, `kernel_release`, `uptime_seconds`) para armazenar informações de sistema operacional e kernel, presentes apenas no payload da VM. A função `_get_primary_ip_from_network` foi adicionada para extrair o IP da interface de rede padrão no payload da VM. O `routes/discovery.py` foi reorganizado com docstring completa documentando os dois contratos JSON aceitos pelo endpoint.

Métricas — alinhamento ao banco e formato real do agente

Os modelos SQLAlchemy `HostDiscovery` e `Metric` foram revisados e alinhados ao schema real do banco de dados definido pela equipe de banco (`init.sql`). O modelo `Metric` foi expandido com os campos `memory_used_mb`, `memory_free_mb`, `memory_total_mb`, `disk_used_mb`, `disk_free_mb` e `disk_total_mb`, todos ausentes na versão anterior. O endpoint POST `/api/metrics` foi atualizado para aceitar o formato real enviado pelo agente (`{ "type": "metrics", "data": { ... } }`).

}), realizando as conversões de bytes para megabytes antes de persistir. O endpoint mantém compatibilidade com o formato legado de campos diretos.

Pendências para a próxima semana

- Criar função `parse_ssh_log(raw_line)` para extrair IP, usuário e status de logs SSH
- Preencher `parsed_data` automaticamente no POST `/api/logs` para logs do tipo auth
- Garantir que logs sem padrão SSH sejam salvos com `parsed_data = NULL`
- Criar consulta SQL que conta tentativas de login falhas por IP nas últimas X horas